

Development Guide

Setup, conventions, and contribution workflow

Hamza Abdul Karim · F23607046 — May 2026

ABSTRACT

This guide describes how to set up a working environment for the NLP search engine, the conventions the codebase follows, and the steps required to add a new ranking model or front-end. The supplied Makefile and Docker image cover the common day-to-day operations.

1. Prerequisites

- Python 3.11 or newer.
- git.
- make (optional, but the supplied Makefile shortcuts are convenient).

2. Getting Started

```
git clone https://github.com/AbdulGhani002/nlp-search-engine.git
cd nlp-search-engine

python -m venv .venv
# Linux/macOS: source .venv/bin/activate
# Windows (PS): .venv\Scripts\Activate.ps1

pip install -e ".[dev]"
```

The editable install lets you edit modules under `src/` and have the changes picked up without reinstalling the package.

3. Common Tasks

Task	Command
Run the test suite	<code>pytest -q</code> (or: <code>make test</code>)
Run a one-off CLI search	<code>python cli.py "your query"</code>
Start the web UI	<code>python app.py</code> (or: <code>make run</code>)
Rebuild the index	<code>python scripts/build_index.py</code>
Rebuild PDF docs	<code>python scripts/build_docs.py</code>
Lint with Ruff	<code>make lint</code>
Format with Black	<code>make format</code>
Build the Docker image	<code>make docker</code>

4. Repository Layout

```
nlp-search-engine/  
app.py Flask UI  
cli.py Argparse CLI  
pyproject.toml Packaging + tool config  
Dockerfile Production container  
Makefile Common commands  
data/sample_docs/ Tiny demo corpus  
docs/ PDFs and SVG diagrams  
examples/ Standalone usage examples  
scripts/ build_index.py, build_docs.py, helpers  
src/ Engine source (every file <200 lines)  
static/ CSS, logos  
templates/ Jinja2 templates  
tests/ Pytest suite  
.github/ CI workflows
```

5. Coding Conventions

- Each source file stays under 200 lines (split when necessary).
- Type hints everywhere; modules use `from __future__ import annotations`.
- Public APIs are exposed through the package facade in `src/__init__.py`.
- Comments only when the why is non-obvious; naming should make the what self-explanatory.

6. Adding a New Ranking Model

The Ranker is intentionally thin so a new scoring function is a small change.

- Implement a class in `src/<your_model>.py` exposing `score(query_terms) -> List[Tuple[int, float]]`.
- Register it in `src/ranker.py`: extend `VALID_MODELS`, add a lazy property, and dispatch in `Ranker.rank`.
- Update `docs/ranking-models.pdf` and add a unit test under `tests/`.

7. Adding a New Front-end

The `SearchEngine` class is framework-agnostic. The minimal sequence is:

```
from src.config import EngineConfig  
from src.search_engine import SearchEngine  
  
engine = SearchEngine(EngineConfig(), model="bm25")  
engine.index_corpus("data/sample_docs")  
results = engine.search("your query", top_k=5)
```

8. Testing Strategy

- Unit tests sit next to the module they exercise in `tests/test_*.py`.
- End-to-end tests drive the `SearchEngine` through realistic queries (see `tests/test_search.py`).
- The suite must remain fast (well under a second locally) so the contributor feedback loop stays painless.

9. Releasing

- Bump `__version__` in `src/__init__.py` and the version in `pyproject.toml`.
- Move the Unreleased section in the changelog under the new version.
- Tag the commit (`git tag -a vX.Y.Z -m "Release vX.Y.Z"`) and push the tag.
- CI runs the test suite on the tagged commit.